Airport Management System

Code Architecture, Build Process, and Presentation Defense Manual

Prepared for	CS 4347 Milestone 2 presentation
Scope	main.py, db.py, queries.py, reservations.py, requirements.txt

What this manual helps you do

- Explain why the project was split into multiple files instead of one large script.
- Walk through exactly what happens when main.py runs and which functions are called next.
- Defend design choices such as validation, reusable database helpers, and transaction-safe reservation updates.
- Describe a clear design → build → test story during the presentation.

1. Executive Overview

This project is a console-based Airport Management System. The code was organized into small, focused modules so that each file handles one major responsibility instead of mixing menu logic, database access, queries, and reservation updates in one place.

The design follows a controller-and-service style structure: main.py acts as the user-facing controller, db.py manages database connectivity, queries.py handles read-only data retrieval, reservations.py handles updates that change the database state, and requirements.txt records the external package needed to run the app.

2. File-by-File Responsibilities

File	Primary role	Used by	Why it is separate
main.py	Program entry point and menu controller	User	Keeps all input/output and menu decisions in one visible place.
db.py	Connection and cursor management	main.py, queries.py, reservations.py	Prevents repeated connection code and centralizes database settings.
queries.py	Read-only SQL and validation helpers	main.py and reservations.py	Separates reporting logic from update logic and keeps SQL retrieval clean.
reservations.py	Reservation create/cancel business logic	main.py	Groups all state-changing operations and seat-count updates together.
requirements.txt	Dependency list	Environment setup	Documents the required connector package for reproducible installation.

3. Dependency Map and Call Flow

```
requirements.txt  -> installs mysql-connector-python

main.py
  |- imports from db.py
  |- imports from queries.py
  `- imports from reservations.py

queries.py
  `- imports get_cursor from db.py

reservations.py
  |- imports get_cursor from db.py
  `- imports validation helpers from queries.py
```

The dependency direction is intentional. Lower-level modules do not depend on main.py. Instead, main.py depends on them. This is good design because the controller should call the service modules, not the other way around.

One-sentence presentation answer

- "We run main.py, and it delegates work downward: db.py handles connections, queries.py reads data, reservations.py changes data, and requirements.txt records the package needed to talk to MySQL."

4. Design Strategy Before Writing Code

The design can be presented as five choices:

- Use one clear entry point so the program is easy to launch and demo.
- Separate database connection code from business logic so credentials and cursor handling are maintained in one place.
- Separate read-only operations from write/update operations to reduce confusion and lower the chance of accidental changes.
- Validate user input early in the controller layer so the database receives clean, predictable values.
- Return simple results such as (success flag, message/data) so the menu layer can print clean feedback to the user.

4.1 Why these separations matter

This structure improves readability, debugging, maintenance, and teamwork. If a connection issue appears, the team looks in db.py. If a retrieval query is wrong, the team looks in queries.py. If reservation logic is wrong, the team looks in reservations.py. This reduces the time needed to locate a bug and explain the system.

5. Step-by-Step Build Order

1. Start from the schema and project goal. The app is not a generic airline system; it is built around the Milestone 2 schema where SEAT stores booked seats and LEG_INSTANCE stores remaining available seats.
2. Create the database access layer first in db.py. This gives every other module a standard way to connect to MySQL and obtain a cursor.
3. Build the read-only query layer in queries.py. Write reusable helpers for existence checks and retrieval functions before writing the menu.
4. Build the reservation update layer in reservations.py. Add validation, seat-availability checks, insert/delete statements, and seat-count updates.
5. Build the controller in main.py last. Connect each menu option to one function from the supporting modules.
6. Add requirements.txt so the environment can be reproduced on another machine.

6. How db.py Was Written and Why

db.py is the database utility layer. Its purpose is to hide repeated connection code from the rest of the project. Instead of every file opening raw MySQL connections manually, the project imports shared helpers from this file.

Function	What it does	Why it exists
prompt_for_password()	Reads password from the user	Keeps password entry centralized and reusable.

get_db_password()	Returns the current stored password	Lets connection code read the password without prompting again.
get_connection()	Creates the MySQL connection	Keeps host, port, user, and database settings in one place.
get_cursor()	Yields connection and cursor, then closes both safely	Avoids connection leaks and repeats less code.
test_connection()	Runs a quick SELECT DATABASE()	Confirms startup is valid before entering the main menu.

A particularly important design choice is the use of a context manager in get_cursor(). That means the code can use “with get_cursor() as (conn, cursor):” and be confident that the cursor closes and the connection closes even if an error happens.

7. How queries.py Was Written and Why

queries.py was written as the read-only data layer. The idea is simple: if an action only retrieves information, it belongs here. This file first defines two low-level helpers—_fetch_all() and _fetch_one()—and then builds readable functions on top of them.

- Existence helpers: flight_exists(), leg_exists(), leg_instance_exists(), seat_is_booked()
- Retrieval helpers: get_flights(), get_flight_legs(), get_fares(), get_leg_instances(), get_leg_instance_detail(), get_booked_seats()
- Business-facing wrappers: safe_get_flights(), safe_get_flight_legs(), safe_get_fares(), safe_get_leg_instances(), safe_show_available_seats_summary()

The “safe_” functions are especially important for presentation. They show that the project was not only written to query data, but also to protect the user experience. These wrappers check whether the requested flight, leg, or date exists before trying to display data, and they convert database exceptions into user-friendly messages.

8. How reservations.py Was Written and Why

reservations.py contains the update logic that changes database state. It handles the two actions that matter most for correctness: making a reservation and canceling one.

The design intent was to keep all state-changing rules together. This is safer than mixing update statements inside main.py because reservation logic needs validation, transaction handling, and seat-count consistency.

- make_reservation() validates customer name, phone number, and seat number.
- It checks that the flight, leg, and dated leg instance exist.
- It checks that the requested seat is not already booked.
- It uses “SELECT ... FOR UPDATE” on LEG_INSTANCE before updating seat counts, which is a strong design choice because it protects the row during the reservation operation.
- It inserts the seat booking into SEAT and decrements LEG_INSTANCE.No_of_avail_seats in the same transaction.
- cancel_reservation() performs the opposite logic: validate, delete from SEAT, increment available seats, commit.

Strong technical point to mention in Q&A

- We did not only insert and delete rows. We also protected seat count consistency by updating LEG_INSTANCE together with the SEAT table, and we locked the leg instance row before decrementing availability.

9. How main.py Was Written and Why

main.py is the controller and the only file the user runs directly. Its job is not to know every SQL detail. Its job is to manage input, call the correct helper function, and print output in a readable form.

A useful way to explain main.py is to break it into four layers inside the same file:

- Startup layer: asks for the MySQL password and tests the connection before showing the menu.
- Utility layer: prompt_valid_date(), print_header(), and print_rows().
- Handler layer: each menu option has a handler function such as handle_show_fares() or handle_make_reservation().
- Control loop: main() keeps the program running until the user chooses Exit.

9.1 Menu option to function map

Option	Handler in main.py	Calls into	Purpose
1	handle_test_connection()	db.test_connection()	Confirm database connectivity
2	handle_show_flights()	queries.safe_get_flights()	Show first 10 flights
3	handle_show_flight_legs()	queries.safe_get_flight_legs()	Show legs for one flight
4	handle_show_fares()	queries.safe_get_fares()	Show fare choices
5	handle_show_leg_instances()	queries.safe_get_leg_instances()	Show scheduled leg instances
6	handle_show_available_seats()	queries.safe_show_available_seats_summary()	Show remaining seats and booked seat rows
7	handle_make_reservation()	reservations.make_reservation()	Create reservation
8	handle_cancel_reservation()	reservations.cancel_reservation()	Cancel reservation
9	Exit path in main()	No external call	Stop program cleanly

10. Runtime Walkthrough: What Happens When main.py Runs

7. Python enters main.py and reaches the line that calls main() when the script is run directly.
8. main() asks for the MySQL password by calling prompt_for_password() from db.py.
9. main() immediately calls test_connection() from db.py. If the connection fails, the user is asked to try again.
10. Once the connection succeeds, the program enters the menu loop.

11. The user selects a menu option; main.py chooses a handler based on that option.
12. The handler collects input, performs basic format checks such as integer leg number or valid date text, and then calls the correct function in queries.py or reservations.py.
13. The lower-level file uses get_cursor() from db.py to talk to MySQL.
14. The result comes back to main.py as data or a message, and main.py prints it.

This flow is important because it proves main.py is the orchestrator, not the data engine. The heavy work happens in the imported modules.

11. Testing Strategy and Demo Checklist

A solid presentation should not stop at implementation. It should explain how the team knew the code was working. A simple but professional test story is below.

11.1 Test categories

- Connection test: launch the program, enter a password, and confirm that the database name is returned.
- Read tests: use options 2 through 6 and confirm that valid flights, legs, fares, instances, and seat summaries display correctly.
- Validation tests: enter a bad date format, nonnumeric leg number, nonexistent flight number, or unavailable seat and confirm that the program prints a clean error message instead of crashing.
- Reservation test: create a reservation and verify that a SEAT row is inserted and available seat count decreases.
- Cancellation test: cancel that reservation and verify that the SEAT row disappears and available seat count increases.
- Repeatability test: run the same menu options multiple times to confirm stable behavior.

11.2 Suggested live demo order

15. Show option 1 first to prove the app is connected to the correct database.
16. Show one read-only path such as option 2 or 4.
17. Show option 6 to explain the schema decision that SEAT stores booked seats, not the whole airplane map.
18. Create one reservation with option 7.
19. Immediately show option 6 again to prove the seat record and remaining count changed.
20. Cancel the same reservation with option 8 and show the reversal.

12. Presentation Defense: Questions You Are Likely to Get

Q: Why did you separate the code into multiple files instead of writing everything in main.py?

A: Because each file has one clear responsibility. That makes the project easier to read, test, maintain, and explain. main.py handles interaction, db.py handles connections, queries.py handles reads, and reservations.py handles updates.

Q: Why does reservations.py import functions from queries.py?

A: Because reservation logic needs validation before it updates the database. Reusing flight_exists(), leg_exists(), leg_instance_exists(), and seat_is_booked() avoids duplicate code and keeps rules consistent.

Q: Why do you use safe_ functions in queries.py?

A: They convert lower-level SQL work into controller-friendly results. main.py receives either success with data or failure with a readable message, so the user sees clean feedback instead of raw exceptions.

Q: Why validate date format in main.py before sending it to the database?

A: Because input should be cleaned as early as possible. That prevents confusing database errors and gives immediate feedback to the user.

Q: What is the purpose of get_cursor() instead of opening a raw cursor each time?

A: It centralizes connection and cursor setup and closes resources safely after use. That reduces repeated code and prevents connection leaks.

Q: How do you keep seat counts consistent when making a reservation?

A: The code checks that the leg instance exists, checks that the seat is free, locks the leg instance row with FOR UPDATE, inserts into SEAT, updates LEG_INSTANCE availability, and commits as one operation.

Q: Why is there no RESERVATION table?

A: Because the final schema follows the textbook-based design used in the project. In this version, SEAT stores each booked seat assignment directly, and LEG_INSTANCE stores the remaining available seat count.

Q: What does requirements.txt do?

A: It documents the external package the project needs so the environment can be recreated quickly on another machine.

13. Final Summary Script for the Presentation

Short closing script

- "We designed the project in layers. First, we separated responsibilities so the program would be easier to build and explain."
- "Then we built the connection layer, the read-only query layer, the reservation update layer, and finally the main menu controller."
- "We tested both normal paths and failure paths, including invalid input and reservation updates."
- "Because of that structure, each part of the code has a clear purpose, and we can explain exactly which file is called and why."